

Chapitre 8 - Fonctions

Solution des exercices

Exercice 1.

```
1 fun main(){
2     println(surfaceRectangle(2.0,4.0))
3 }
4
5 fun surfaceRectangle(longueur: Double, largeur:Double): Double {
6     return (longueur * largeur)
7 }
```

Exercice 2.

```
1 fun main(){
2     println(ligneCar(3,"allo"))
3 }
4
5 fun ligneCar(n: Int, ca: String): String {
6     return (ca.repeat(n))
7 }
```

Exercice 3.

```
1 fun main(){
2     println(surfCercle(2.5))
3 }
4
5 fun surfCercle(R: Double): Double {
6     return (3.1416 * R * R)
7 }
```

Exercice 4.

```
1 fun main(){
2     println(volBoite(5.2, 7.7, 3.3))
3 }
4
5 fun volBoite(x1: Double, x2: Double, x3:Double): Double {
6     return (x1 * x2 * x3)
7 }
```

Exercice 5.

```
1 fun main(){
2     println(maximum(2,5,4))
3 }
4
5 fun maximum(n1: Int, n2: Int, n3: Int): Int {
6     if (n1 > n2 && n1 > n3){
7         return n1
8     }
9     else if (n2 > n1 && n2 > n3){
10        return n2
11    }
12    else{
13        return n3
14    }
15 }
```

Exercice 6.

```
1 fun main(){
2     val serie = arrayOf<Int>(5, 8, 2, 1, 9, 3, 6, 7)
3     println(indexMax(serie))
4 }
5
6 fun indexMax(liste: Array<Int>): Int {
7     return (liste.indexOf(liste.max()))
8 }
```

Exercice 7.

```
1 fun main(){
2     println(inverse("allo"))
3 }
4
5 fun inverse(ch: String): String {
6     var chaineInverse = ""
7
8     for (caract in ch) {
9         chaineInverse = caract + chaineInverse
10    }
11    return (chaineInverse)
12 }
```

Exercice 8.

```
1 fun main(){
2     println(compteMots("Bonjour le monde"))
3 }
4
5 fun compteMots(ph: String): Int {
6     return (ph.split(" ").size)
7 }
```

Exercice 9.

```
1 fun main(){
2     println(nomMois(4))
3 }
4
5 fun nomMois(n: Int): String {
6     val mois =
7     arrayOf<String>("Janvier", "Février", "Mars", "Avril", "Mai", "Juin", "...")
8     return (mois[n-1])
9 }
```

Exercice 10.

```
1 fun main(){
2     println(nomJour(0))
3 }
4
5 fun nomJour(n: Int): String {
6     val jours = arrayOf<String>("Lundi", "Mardi", "Mercredi",
7     "Jeudi", "Vendredi", "Samedi", "Dimanche")
8     return (jours[n])
9 }
```

Exercice 11.

```
1 import java.time.LocalDate
2
3 fun main(){
4     println(premierJour(2023, 11))
5 }
6
7 fun premierJour(annee: Int, mois: Int): Int {
8     return (LocalDate.of(annee, mois, 1).getDayOfWeek().getValue() - 1)
9 }
```

Exercice 12.

```
1 import java.time.LocalDate
2
3 fun main(){
4     println(nbJour(2023, 11))
5 }
6
7 fun nbJour(annee: Int, mois: Int): Int {
8     var nombreJourDansLeMois: Int
9
10    if (mois == 2){
11        if (LocalDate.of(annee, mois, 1).isLeapYear())
12            nombreJourDansLeMois = 29;
13        else
14            nombreJourDansLeMois = 28;
15    }
16    else{
17        if (mois == 1 || mois == 3 || mois == 5 || mois == 7 || mois == 8 ||
18        mois == 10 || mois == 12)
19            nombreJourDansLeMois = 31;
20        else
21            nombreJourDansLeMois = 30;
22    }
23    return (nombreJourDansLeMois)
24 }
```